Course Notes

Lecture 02 — Sensors, AD/DA Conversion and Interrupts

Communication Technologies for the Internet of Things

Academic year 2026–2027

Catarina Silva

ESTGA, University of Aveiro

Text to support independent study

Contents

How to Use These Notes **2**

1 Commentary on the Slides **2**

2. Agenda . . . 2
3. From Theory to Practice . . . 3
4. What Is a Sensor (1/2) . . . 4
5. What Is a Sensor (2/2) . . . 5
6. Why Convert Analogue into Digital (1/2) . . . 6
7. Why Convert Analogue into Digital (2/2) . . . 7
8. Sampling (1/2) . . . 8
9. Sampling (2/2) . . . 9
10. Resolution and Quantisation Error . . . 10
11. In Practice: the Arduino Uno ADC . . . 11
12. Why Interrupts? . . . 12
13. Hardware Interrupts (1/2) . . . 13
14. Hardware Interrupts (2/2) . . . 14
15. Software Interrupts (1/2) . . . 15
16. Software Interrupts (2/2) . . . 16
17. Good Practice in an ISR . . . 17
18. The Equipment for This Session (1/2) . . . 18
19. The Equipment for This Session (2/2) . . . 19
20. Assembly Precautions . . . 20
21. Visual Programming: No-Code and Low-Code . . . 21
22. Node-RED . . . 22
23. Structure of the Practical Session (2h) . . . 23
24. Lesson Summary . . . 24

Final Check **25**

References **25**

How to Use These Notes

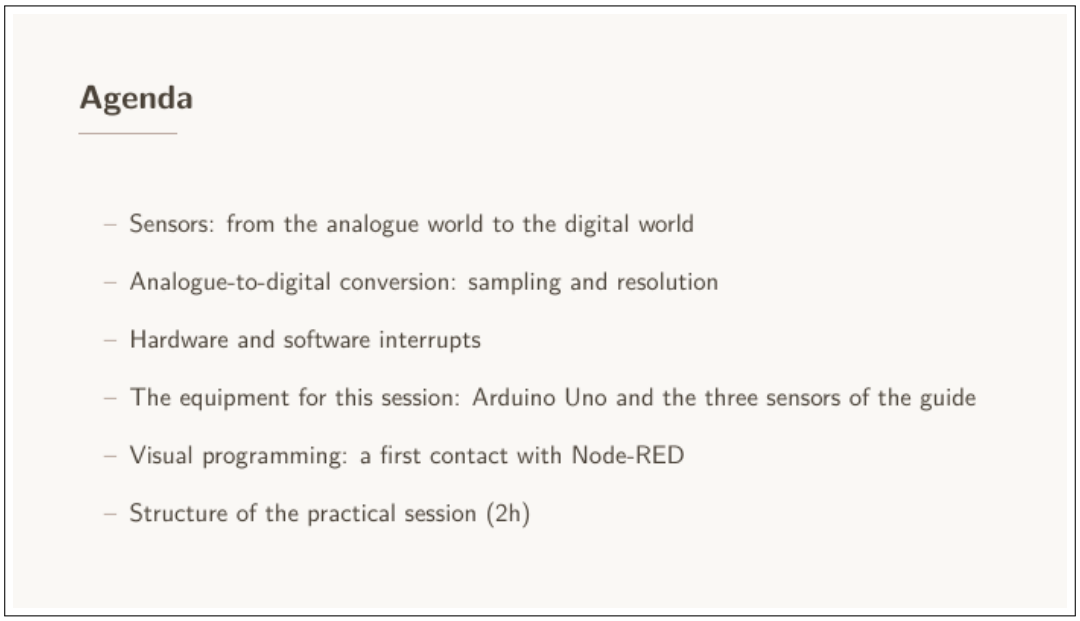
These notes follow the lecture slide by slide. Each page reproduces a slide and states, below it, what should be taken from it.

Reading them does not replace the lecture or the slides. They are meant for later study, once the student has heard the lecture and wants to consolidate, check that the essentials were understood or prepare for assessment. The comments make explicit links between slides that remain implicit in the oral presentation, and point out where the material is commonly misunderstood.

Whenever a comment poses a question, that question is the verification criterion: if you cannot answer it, go back to the slide before moving on.

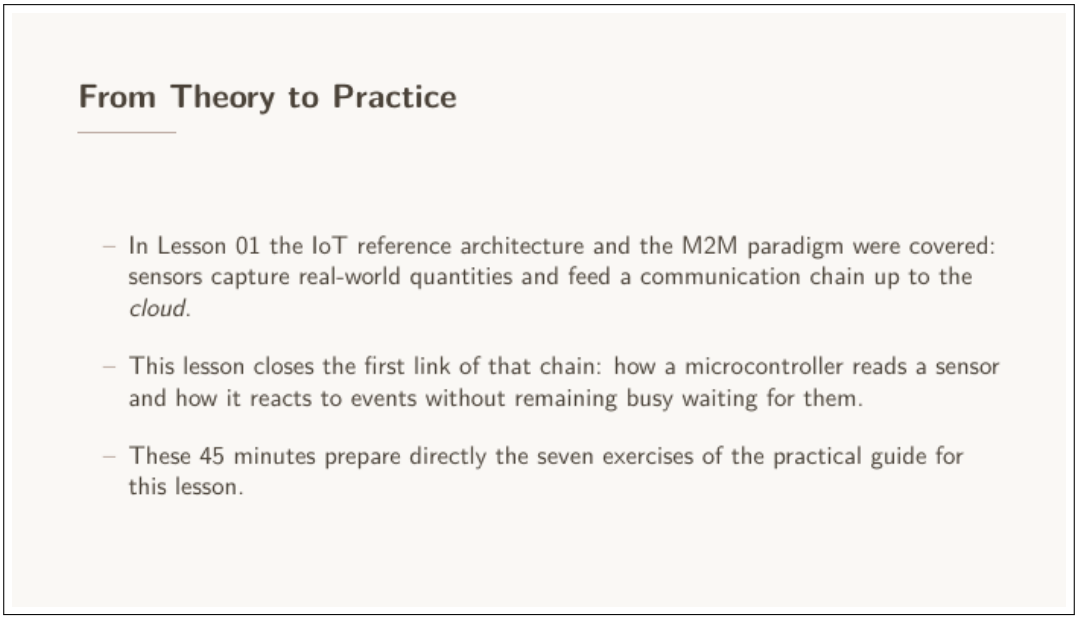
1. Commentary on the Slides

Slide 2. Agenda



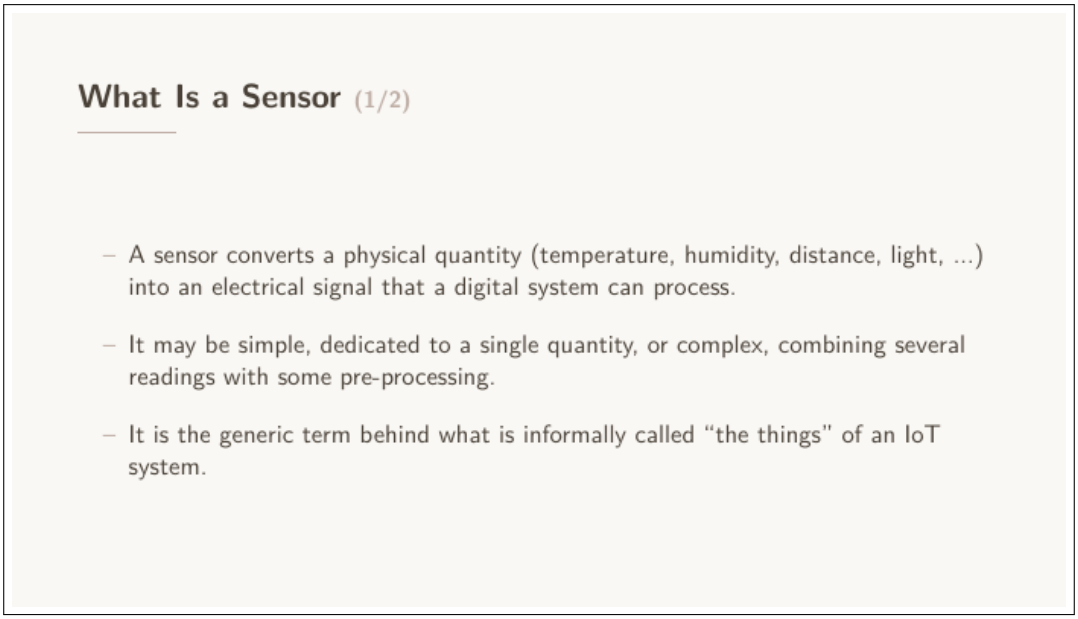
The path through the lecture. The talk has three blocks. The first deals with the acquisition of physical quantities, the second with responding to events, the third with visual programming tools. All three converge on the same point: preparing the lab session that follows immediately. This lecture is not self-contained, it is instrumental.

Slide 3. From Theory to Practice



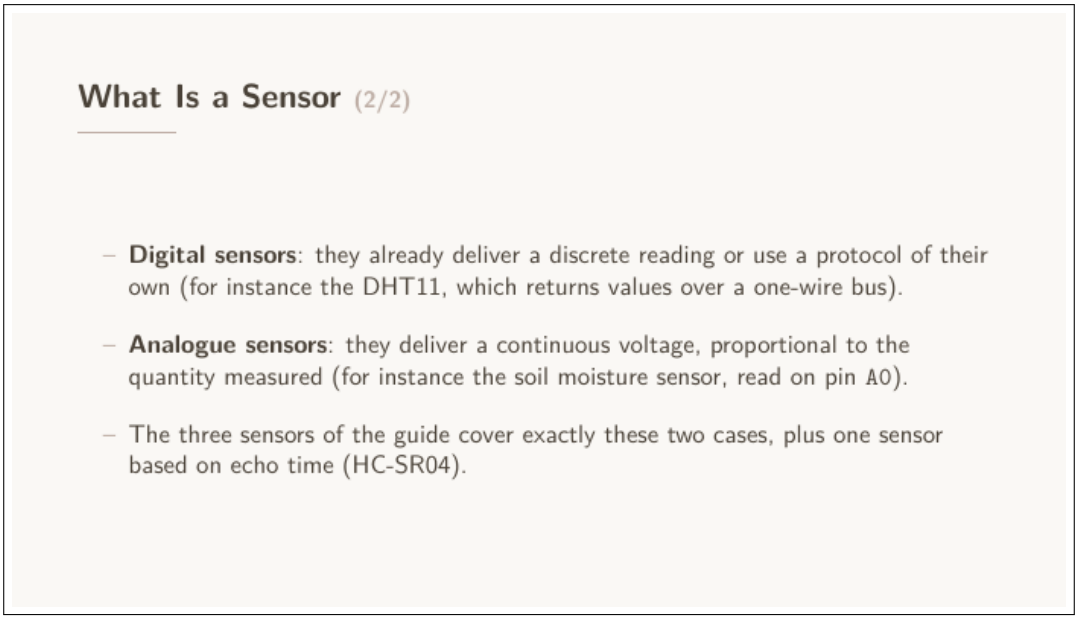
What has been covered and what is missing. In Lecture 01 the reference architecture and the machine-to-machine paradigm were studied. It was established that sensors capture quantities and feed a communication chain. What remains is to close the first link of that chain: how a microcontroller actually reads a sensor, and how it reacts to events without being permanently busy waiting for them.

Slide 4. What Is a Sensor (1/2)



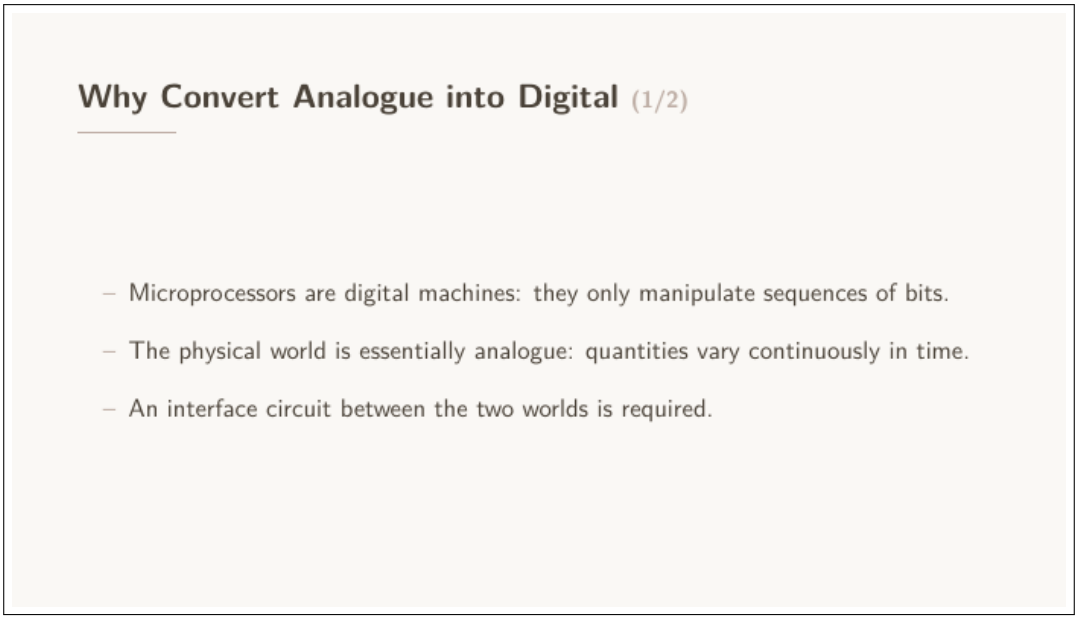
What a sensor is. A sensor converts a physical quantity into an electrical signal that a digital system can process. It may be simple, dedicated to one quantity, or complex, combining readings and some preprocessing. It is the technical term behind what are informally called the things of an IoT system.

Slide 5. What Is a Sensor (2/2)



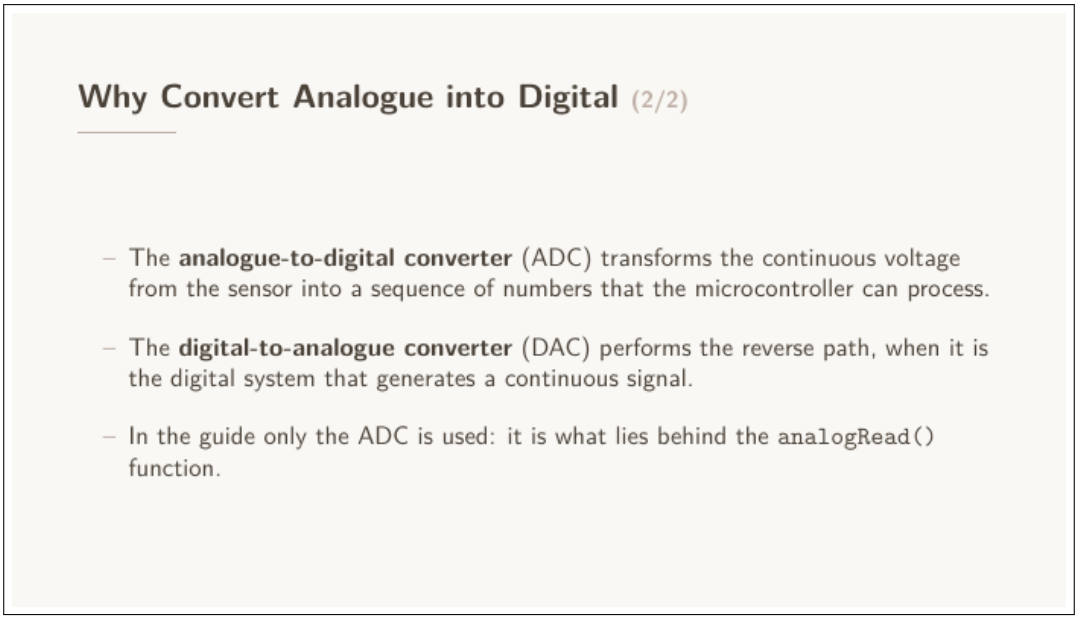
Digital and analogue. Digital sensors deliver a discrete reading or use a protocol of their own. Analogue ones deliver a continuous voltage, proportional to the quantity measured. This distinction is not academic: it determines whether a direct read function is enough or a dedicated library is needed. The three lab sensors cover both cases, plus a third that measures echo time.

Slide 6. Why Convert Analogue into Digital (1/2)



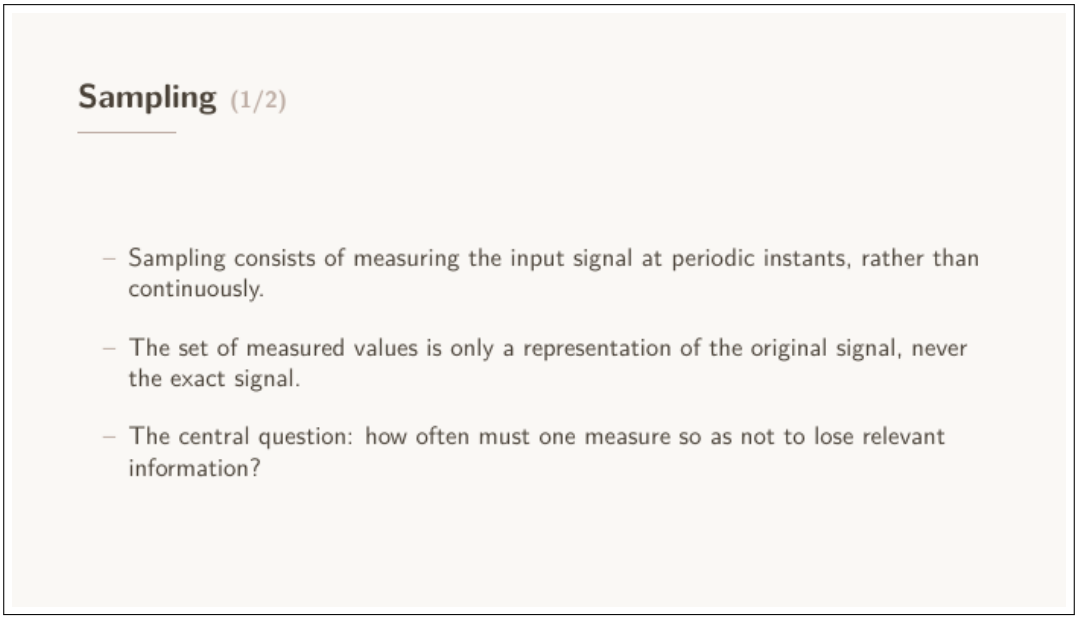
The basic incompatibility. Microprocessors are digital machines and only handle sequences of bits. The physical world is analogue and varies continuously. An interface circuit is needed between the two. This slide states the problem; the next presents the solution.

Slide 7. Why Convert Analogue into Digital (2/2)



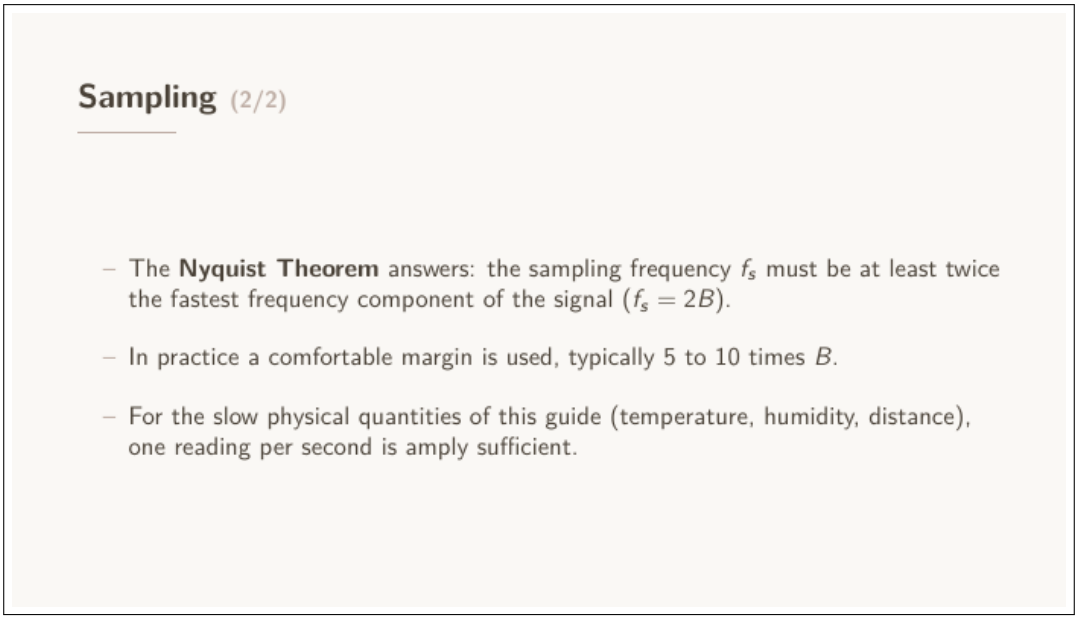
ADC and DAC. The analogue-to-digital converter turns the continuous voltage into a sequence of numbers. The digital-to-analogue converter does the reverse. Only the first is used in the lab. It is what lies behind the Arduino's analogue read function, which is worth remembering for the soil moisture sensor exercise.

Slide 8. Sampling (1/2)



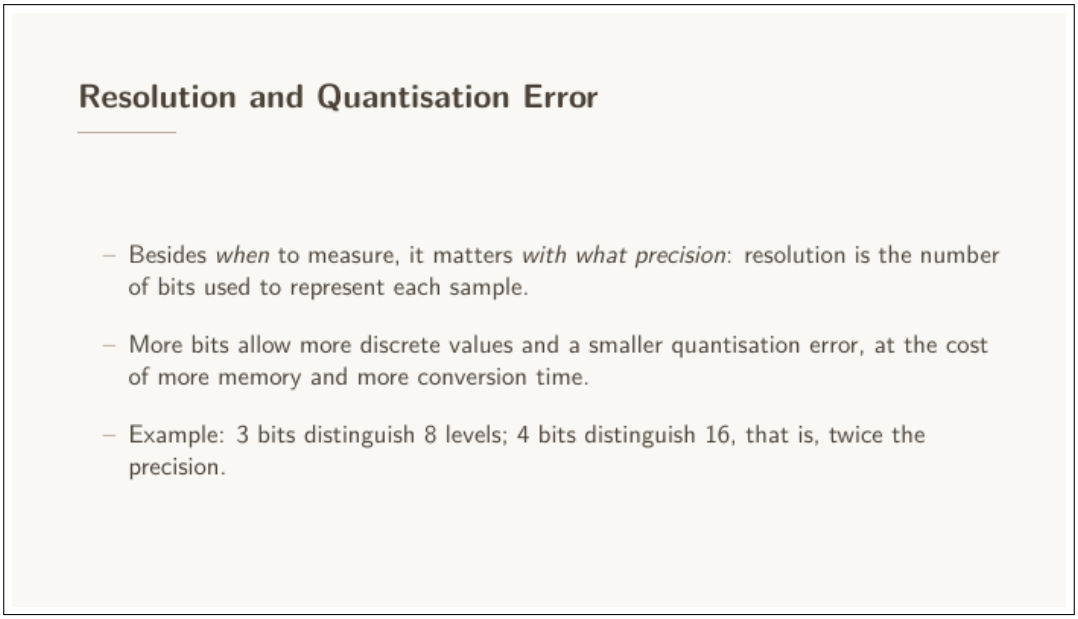
What sampling is. Sampling consists of measuring the signal at periodic instants, not continuously. The set of measured values is a representation of the original signal, never the exact signal. The central question follows: how often must one measure so as not to lose relevant information.

Slide 9. Sampling (2/2)



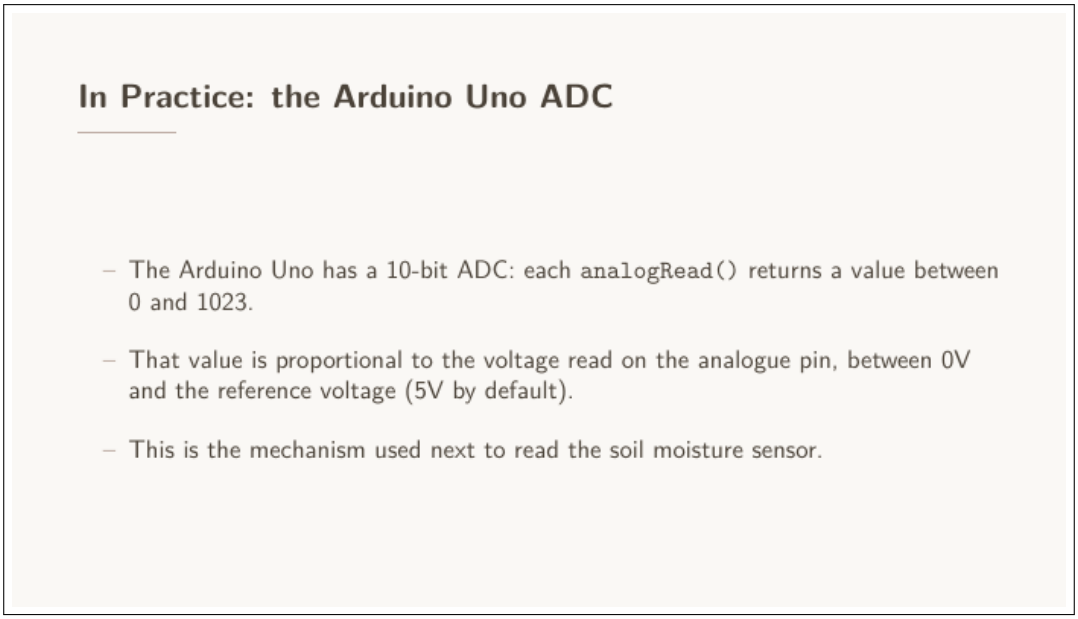
The Nyquist criterion. The sampling frequency has to be at least twice the fastest component of the signal. In practice a margin of five to ten times is used. For the slow quantities of the lab, temperature, humidity and distance, one reading per second is more than enough. The criterion matters when fast quantities are measured, such as sound or vibration.

Slide 10. Resolution and Quantisation Error



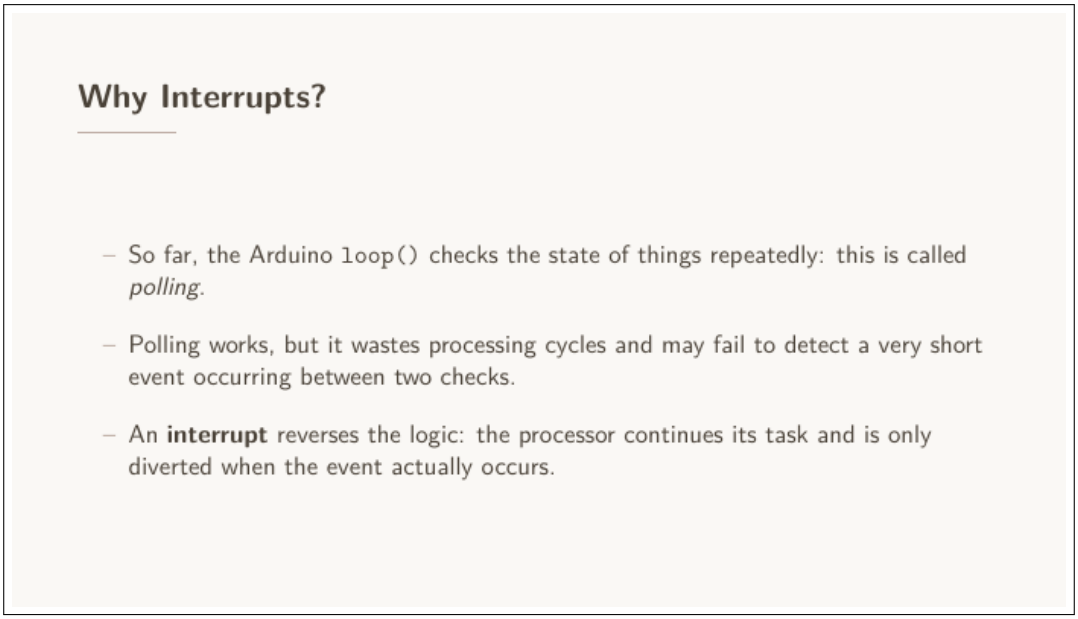
How many bits per sample. Besides when to measure, how precisely matters. Resolution is the number of bits used to represent each sample. More bits mean more discrete levels and smaller quantisation error, at the expense of memory and conversion time. Three bits distinguish eight levels, four bits distinguish sixteen.

Slide 11. In Practice: the Arduino Uno ADC



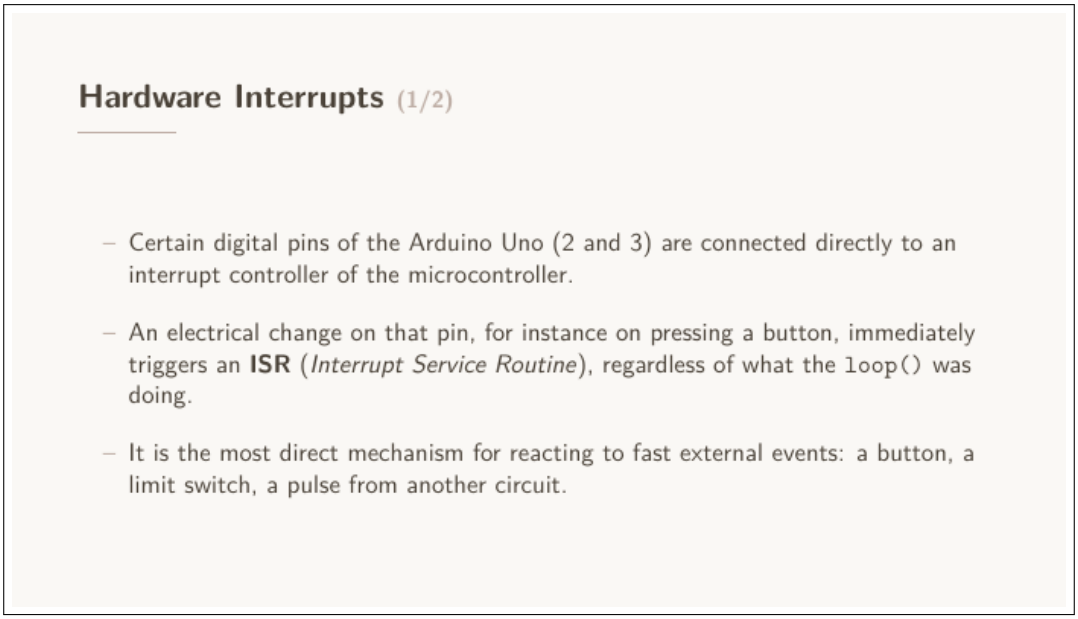
The Arduino's concrete case. The Arduino Uno has a 10-bit converter, so each reading returns a value between 0 and 1023. That value is proportional to the voltage read, between zero volts and the reference voltage. When a number in that range shows up in the lab, this is the explanation. Two to the power of ten is 1024 possible values.

Slide 12. Why Interrupts?



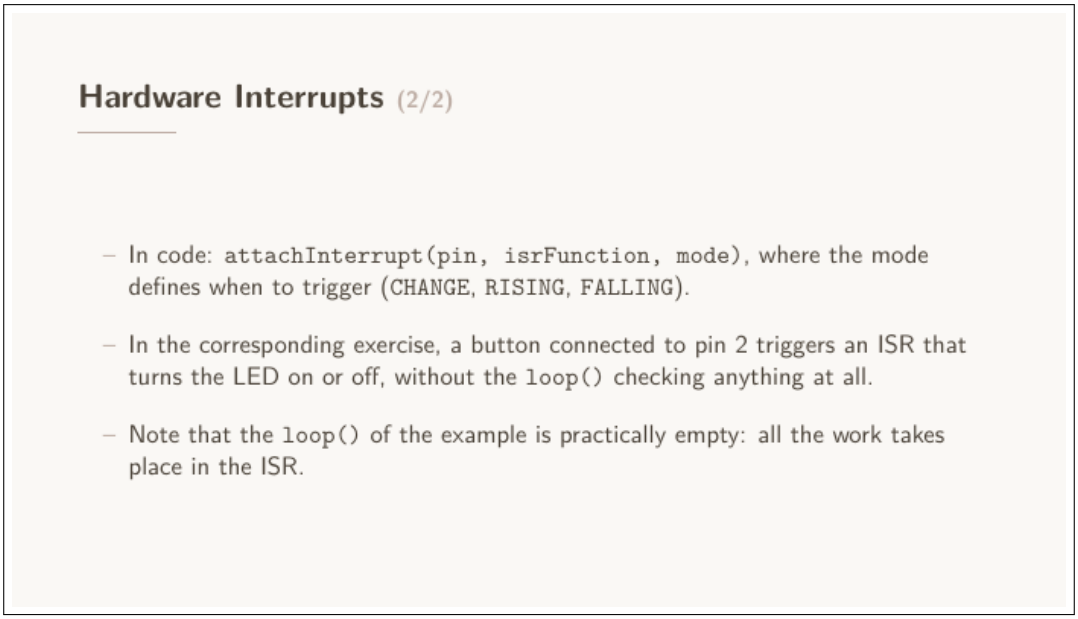
Polling and its cost. So far, the program's main loop repeatedly checks the state of things. This approach is called polling. It works, but wastes processing cycles and may miss a short event that occurs between two checks. The interrupt inverts the logic: the processor carries on with its task and is only diverted when the event occurs.

Slide 13. Hardware Interrupts (1/2)



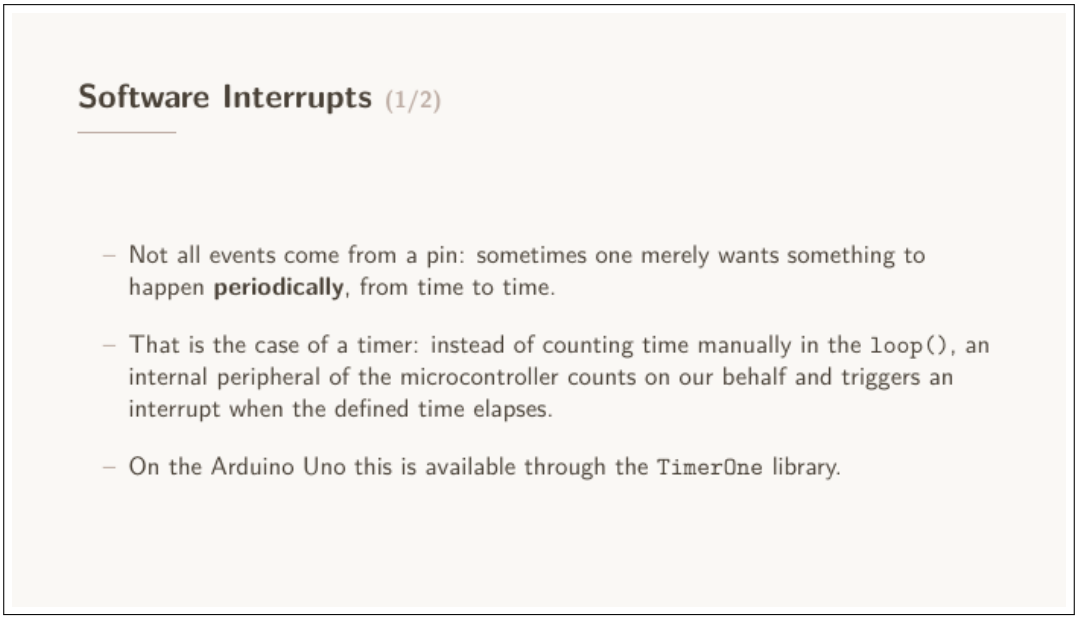
Interrupts tied to pins. Certain digital pins of the Arduino Uno, 2 and 3, are connected to an interrupt controller. An electrical change on that pin immediately triggers a service routine, regardless of what the main loop was doing. It is the most direct mechanism for reacting to fast external events.

Slide 14. Hardware Interrupts (2/2)



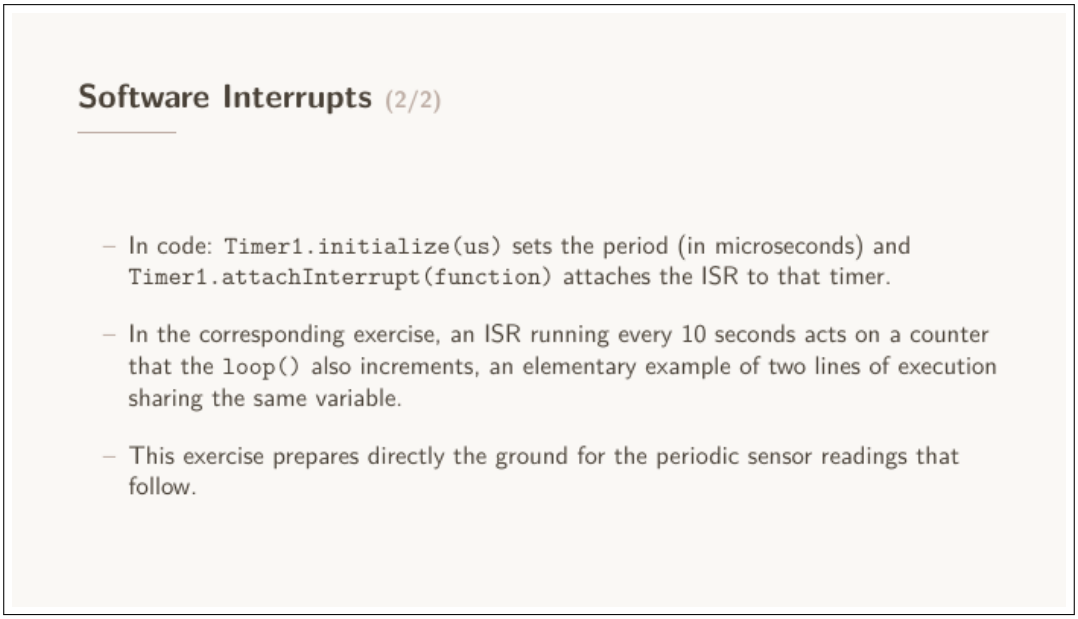
What to observe in the exercise. The attach function takes the pin, the routine and the mode, which defines when to trigger. The detail to observe in the lab is that the main loop is practically empty. All the behaviour lives in the service routine, and that is visible in the code.

Slide 15. Software Interrupts (1/2)



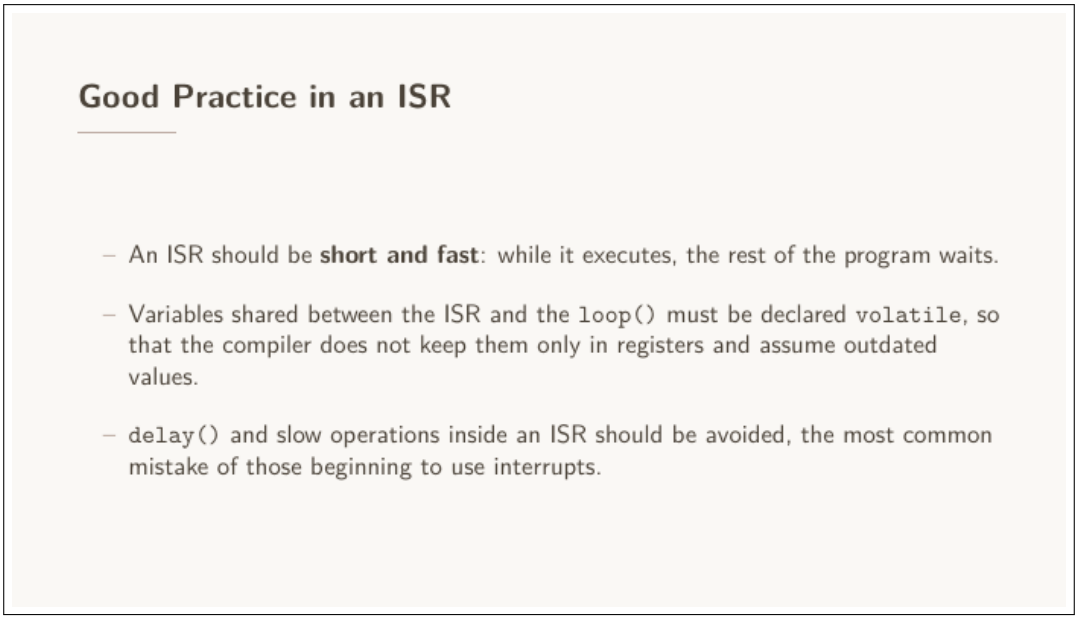
When the event does not come from outside. Not all events come from a pin. Sometimes one just wants something to happen periodically. An internal peripheral counts time and triggers an interrupt when the period runs out, which dispenses with manual counting in the main loop.

Slide 16. Software Interrupts (2/2)



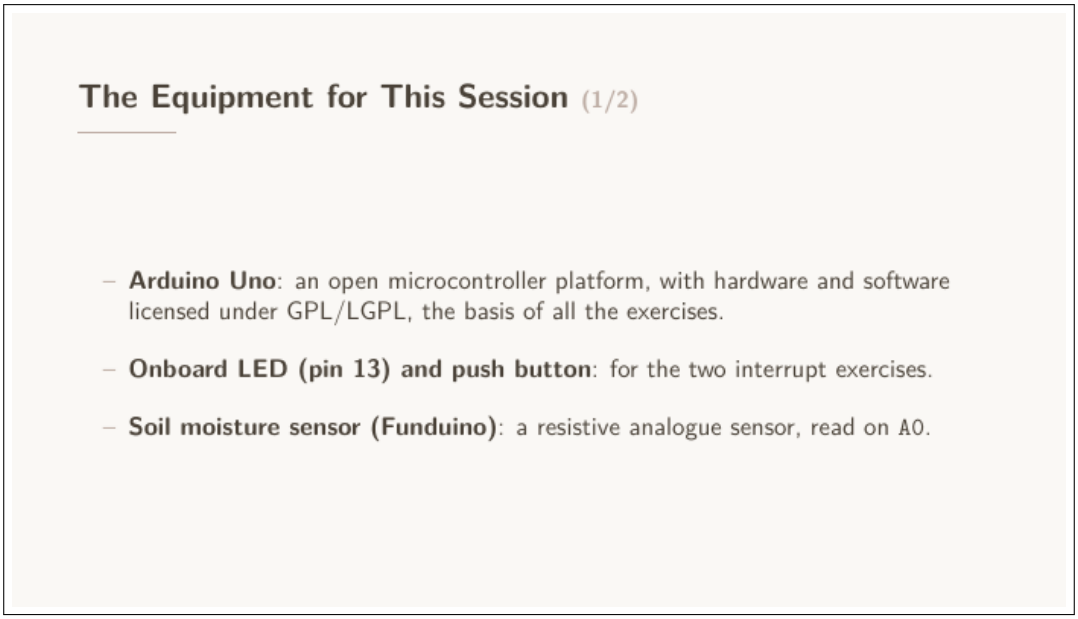
Two threads of execution on the same variable. The timer library lets you define the period and attach the routine. In the exercise, a periodic routine subtracts from a counter that the main loop increments. It is a deliberately simple example of two threads of execution acting on the same variable, and prepares the next slide.

Slide 17. Good Practice in an ISR



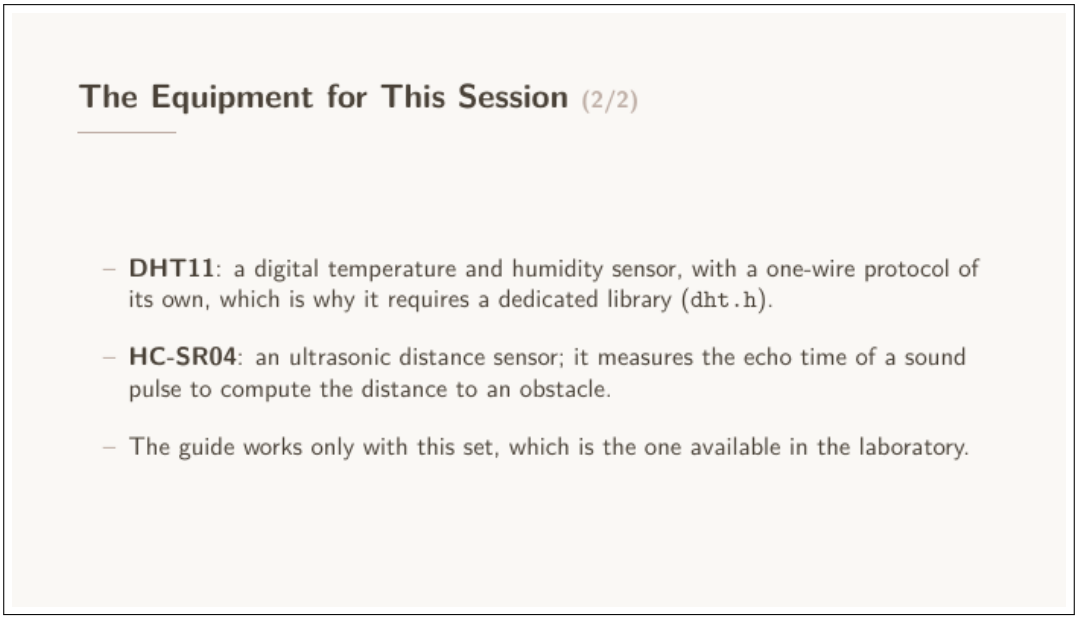
The three rules. A service routine should be short, because while it runs the rest of the program waits. Shared variables should be declared volatile, so that the compiler does not assume stale values kept in registers. Delays and slow operations should be avoided. The third rule is the most common mistake of beginners, and is worth keeping in mind in the lab.

Slide 18. The Equipment for This Session (1/2)



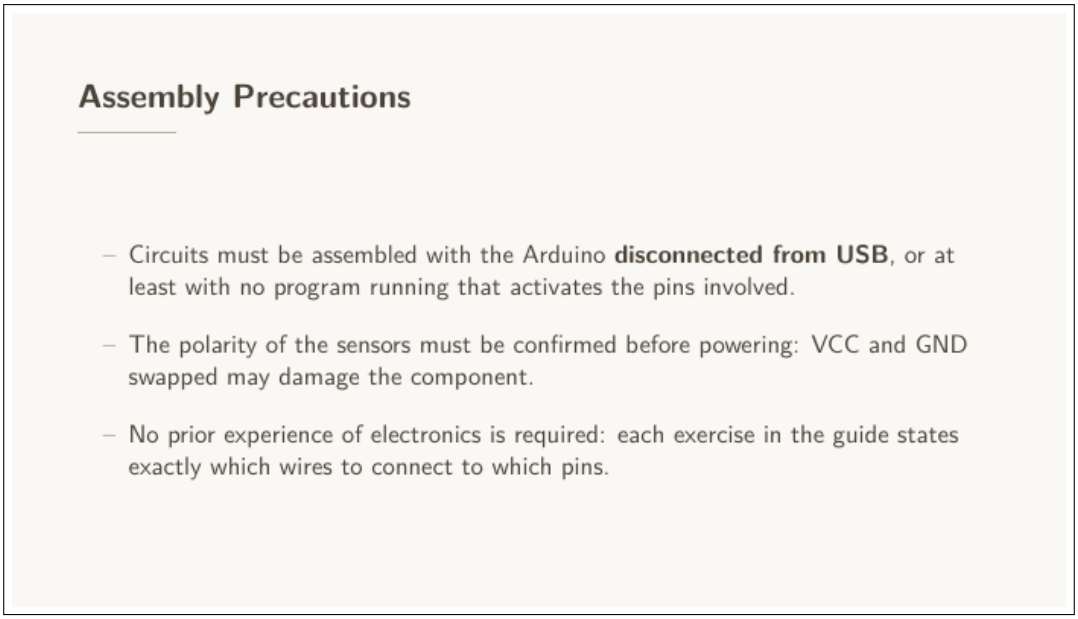
The equipment, first part. The Arduino Uno is an open microcontroller platform. The built-in LED on pin 13 and the push button serve the two interrupt exercises. The soil moisture sensor is resistive and analogue.

Slide 19. The Equipment for This Session (2/2)



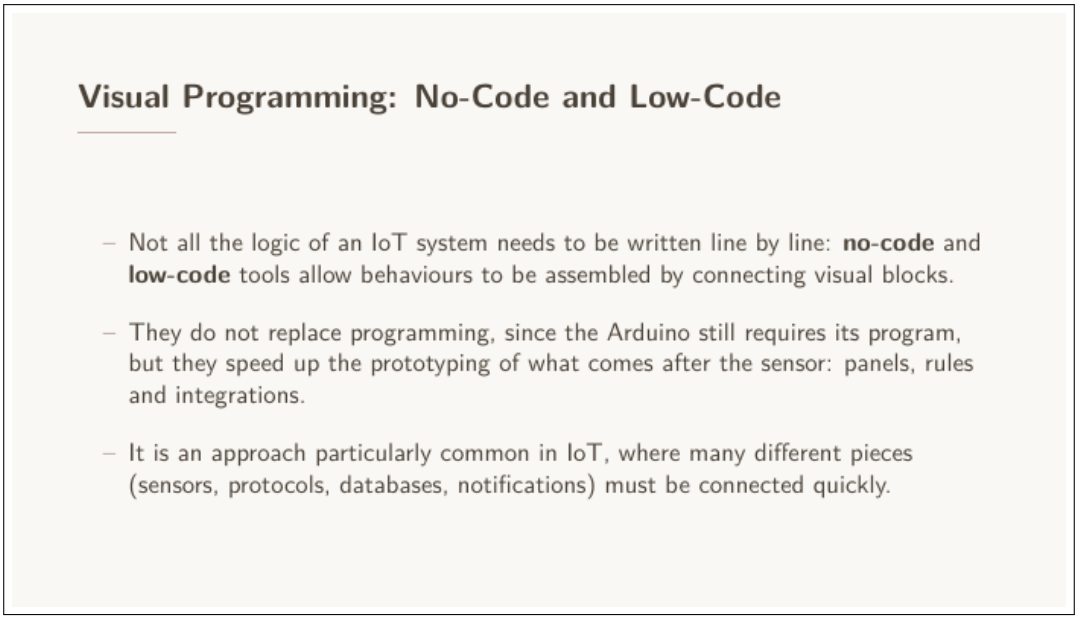
The equipment, second part. The DHT11 is digital and communicates through a protocol of its own on a single wire, which is why it requires a dedicated library. The HC-SR04 measures distance by the echo time of a sound pulse. These two sensors will reappear in the next lecture, when the reading starts to be transmitted by radio.

Slide 20. Assembly Precautions



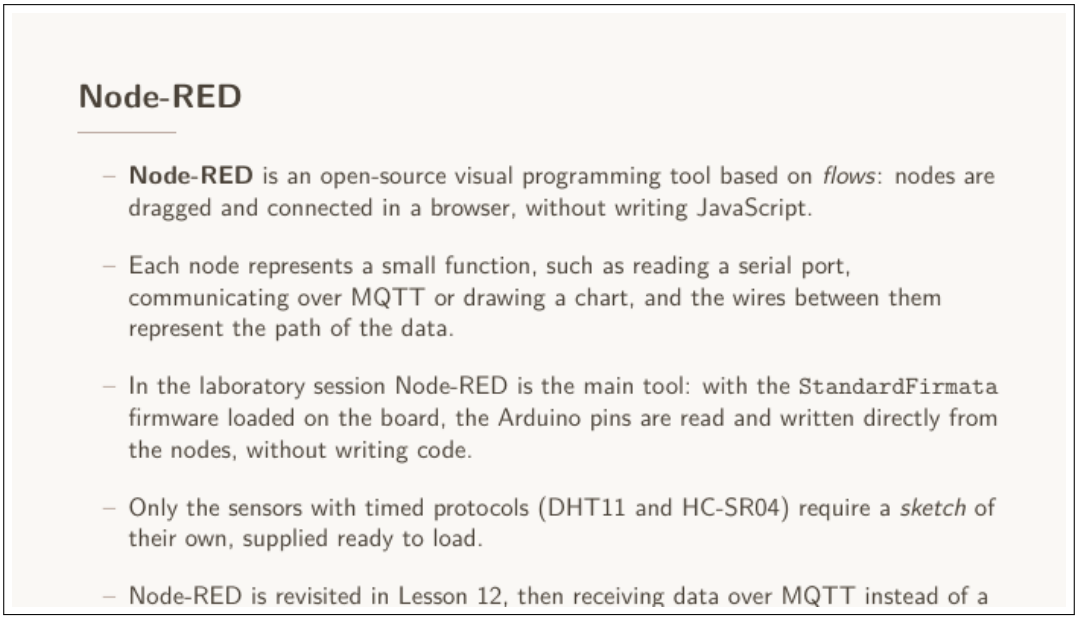
Before powering on. Circuits should be assembled with the board off. Polarity should be confirmed before powering, because swapped supply and ground damage the component. No prior experience of electronics is required: the guide says exactly which wires to connect to which pins.

Slide 21. Visual Programming: No-Code and Low-Code



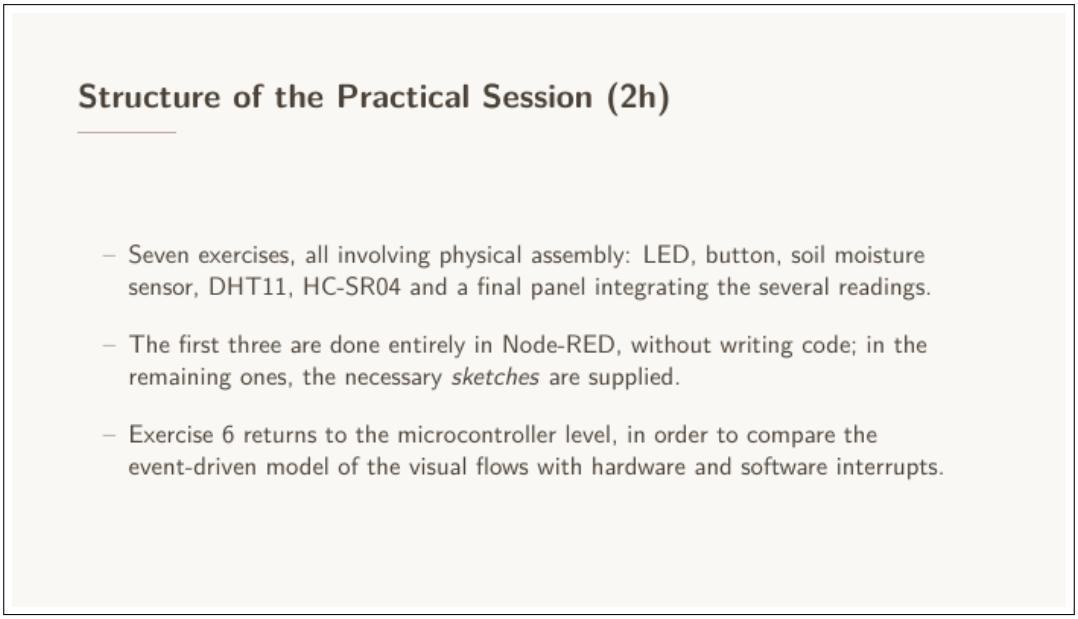
Programming without writing code. Not all the logic of an IoT system needs to be written line by line. Visual programming tools let behaviours be assembled by connecting blocks. They do not replace programming, and the Arduino still needs its program. They speed up prototyping of what comes after the sensor: dashboards, rules and integrations.

Slide 22. Node-RED



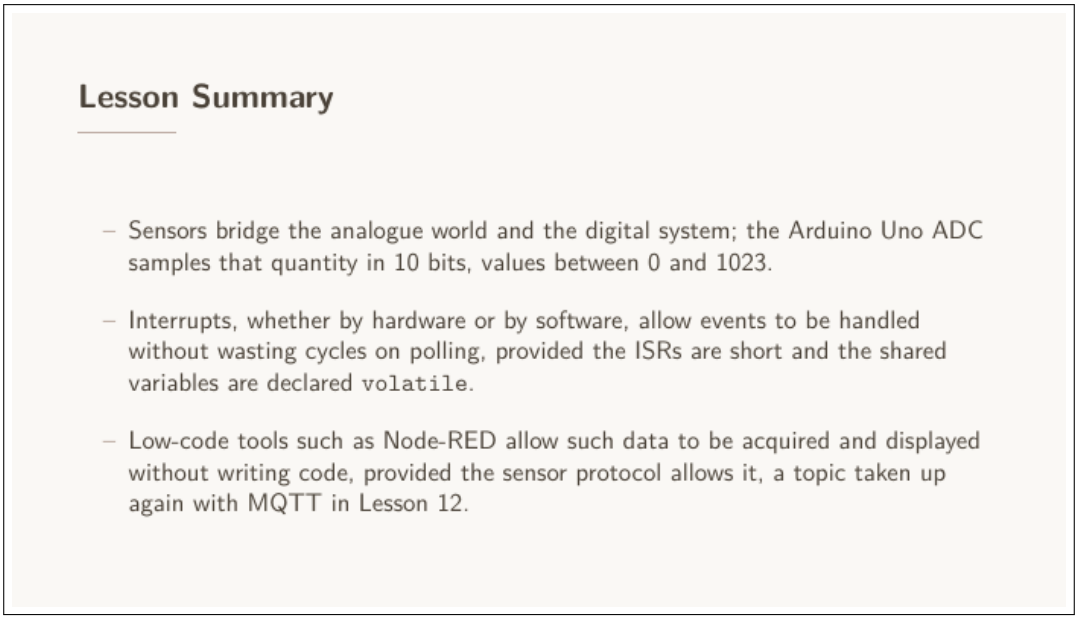
What Node-RED is. Node-RED is an open-source flow-based visual programming tool. Each node represents a small function and the wires between them represent the path of the data. With the appropriate generic firmware loaded on the board, pins can be read and written directly from the nodes. This tool will come back in Lecture 12, then with a messaging protocol instead of a direct connection.

Slide 23. Structure of the Practical Session (2h)



How the session runs. There are seven exercises, all with physical assembly. The first three are done entirely in Node-RED, without writing code; in the rest, the necessary programs are provided. The sixth exercise returns to the microcontroller level, to compare the event-driven model of visual flows with hardware and software interrupts. That is where the two blocks of this lecture meet.

Slide 24. Lesson Summary



Lecture summary. Sensors bridge the analogue world and the digital system, and the Arduino's converter samples at 10 bits, between 0 and 1023. Interrupts let the system react to events without wasting cycles, provided routines are short and shared variables are declared volatile. Low-code tools make it possible to acquire and visualise that data without programming, provided the sensor protocol allows it.

Final Check

Before considering the material studied, check that you can answer the following without looking at the slides.

1. Distinguish an analogue sensor from a digital one and state the consequence of that distinction for reading.
2. Justify why a reading from the Arduino Uno's converter lies between 0 and 1023.
3. State the Nyquist criterion and explain why it is not restrictive for the lab quantities.
4. Compare polling and interrupts in terms of processing consumption and the risk of missing events.
5. Explain the role of the word volatile and what happens if it is omitted.
6. State the three rules to observe in an interrupt service routine.
7. Explain under what circumstances a low-code tool does not dispense with writing a program on the board.

References

- Gonçalves, P. *Apresent1 — Introduction* (sensors and AD/DA conversion block) and *guide 1 — arduino*, TCIoT, previous editions.
- Arduino documentation on external interrupts: `arduino.cc/reference`
- Node-RED documentation: `nodered.org/docs/`